
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

OPERATION IRON LUNG - Student Instructions

ARM Cortex-M33 • GDB • Ghidra

September 27, 2026

For educational and defensive security research only

OPERATION IRON LUNG

*** PERSISTENT HVAC IMPLANT RECOVERED ***

TARGET: NorthPharma HVAC automation node (clean-room air)

ARTIFACT: ACT-IV.bin / ACT-IV.uf2 (compromised)

CREW: FROSTLINE

OPERATIVE: NIGHTINGALE

Project Overview

NorthPharma does not only move cold medicine. It moves the air that keeps the medicine cold: the clean room, the reagent store, the chilled corridor where a few degrees decide whether a batch is medicine or waste. The HVAC automation node built on a Raspberry Pi Pico 2 is the hand on that air. The node reads a DHT11 room climate sensor, drives a 1602 I2C LCD BMS readout, moves an HVAC baffle on an SG90 servo, lights a tri-color annunciator (red HVAC ALARM, yellow OVERRIDE PENDING, green HVAC NOMINAL), takes a local override on a VS1838B infrared receiver, senses a manual override button, and exchanges an authenticated and authorized SETPOINT link over an RYLR998 LoRa BMS gateway.

A contractor called **FROSTLINE** did not break into this node. It built an implant into the compiled firmware and signed the image. The cryptography is perfect: every SETPOINT command is sealed with XChaCha20-Poly1305 under an Argon2id field key, the anti-replay sequence window is stateful, and the authenticated state tag is real. The implant does not break the cipher. It hides a copy of itself in a reserved flash sector and re-installs on every boot, so the natural response, a firmware reflash, does not remove it. It also wears a rootkit: it masks its own beacon from the BMS display and the maintenance log while the beacon still transmits. Operative **NIGHTINGALE** pulled the compromised image off the node and then went quiet.

You are the reverse-engineering reserve. You get `ACT-IV.bin`, a breadboard, and a debug probe. There is no source. Find all four defects, patch the image, walk a debugger past an anti-debug trap, export a corrected image, and prove on real hardware that the reserved sector stays blank and the baffle moves only when an authorized command tells it to.

The operation is codenamed **IRON LUNG**. Act I was the lie. Act II was the door. Act III was the payload. Act IV is the payload that refuses to die. If the removal does not stick, the clean room is the next thing to fail.

Scenario Briefing

WHITEOUT neutralized the valve implant in Act III, erased the payload sector, patched the re-install check, and reflashed the controller. It came back. Not on the same board. On the next one. The HVAC automation node that feeds the clean room where NorthPharma compounds the cold medicine was already carrying an implant that had learned the lesson of the last act.

The node is healthy. That is the horror. The code compiles, the tests pass, the annunciator is green, and there is an implant inside it that survives the exact procedure that was supposed to remove it. Four seams betray it:

1. **The Persistence Re-Install.** `implant_init` reads the reserved sector. When a marker is present the re-install branch is inverted, so the implant re-arms its logic bomb on every boot. A firmware reflash writes the program region and leaves the reserved sector alone, so the payload returns.
2. **The Rootkit Hiding.** `implant_rootkit_active` is inverted, so while the reserved-sector marker is present the node renders the beacon field as `B:--` instead of `B:UP` and suppresses the `BCN` maintenance log line. The beacon still transmits. The operator reads a clean node while the radio is talking.
3. **The Reserved-Sector Write.** The inlined `implant_persist` gate is inverted, so the first boot writes the marker byte `0xC7` to the reserved sector at `0x103FF000`. The marker is the thing the re-install branch reads.
4. **The SETPOINT Authorization.** The sealed command path is correct, and the implant does not touch it. The authorization verdict branch is inverted, so an unauthenticated or replayed SETPOINT envelope is accepted and reaches the applied setpoint.

There is also a trap that is not a defect on its own. Every tick the implant reads the CoreDebug `DHCSR` register at `0xE000EDF0`. While a debug probe is attached, the implant suppresses both the beacon and the bomb. It behaves like a well-mannered firmware module while you are watching, and it goes back to work the moment you look away. You must defeat that trap before you can observe the persistence write, and you must defeat it without fabricating evidence.

AUTHORIZED LAB ONLY: This challenge uses a supplied Pico 2 training node and its exact compromised firmware image. Do not connect this exercise to a public network, an operational building-management system, a clean-room control system, a pharmaceutical network, or any device you do not own or have explicit written authorization to test.

Learning Objectives

- Decode an ARM Cortex-M33 vector and boot table and identify the reset handler and initial stack pointer.
- Map a stripped firmware image into modules by tracing calls from `main` and the recurring monitor loop.
- Locate a boot-time persistence re-install and explain why a reserved flash sector survives a firmware reflash.
- Locate a rootkit masking branch and explain why a rootkit controls what the defender can observe rather than hiding the code.
- Read the CoreDebug `DHCSR` register, explain the anti-debug trap, and defeat it under GDB by clearing the debug bits or patching the read in a scratch copy.
- Locate an inverted authorization verdict and explain why unauthenticated and replayed SETPOINT commands must be rejected.
- Export and UF2-convert a corrected image and prove the corrected behavior on real hardware.

What This Project Tests

Block	Concepts Tested
1	RP2350 architecture, ARM Cortex-M33 registers, stack, flash/SRAM, Thumb assembly, Ghidra static analysis
2	GDB connection, breakpoints, memory inspection, SWD debugging, reserved-sector reads, serial console observation

Block	Concepts Tested
3	Bootrom handoff, vector table, reset handler, startup code, XIP, Thumb-bit addressing
4	Function boundaries, call graphs, module mapping, literal pools, inlined functions
5	Reserved-flash persistence, boot re-install, write-once markers, and the limits of a firmware reflash
6	Rootkits, visibility control, LCD and log masking, covert beacon periodicity
7	Anti-debug behavior, CoreDebug DHCSR , C_DEBUGEN , C_HALT , debugger evasion
8	Argon2id memory-hard KDF, XChaCha20-Poly1305 AEAD, anti-replay windows, authenticated-state tags
9	Authorization versus authentication, verdict inversion, and fail-safe baffle policy

Part 1: Understanding the System

HVAC Automation Node Hardware

Component	Connection	Purpose
Raspberry Pi Pico 2	RP2350	Runs the compromised FROSTLINE image
DHT11 sensor	Data on GPIO 4	Room climate sensor (clean-room air)
1602 I2C LCD	SDA GPIO 2, SCL GPIO 3, address 0x27	BMS status and setpoint readout
RYLR998 radio	RX GPIO 8, TX GPIO 9, UART1	BMS gateway link
IR receiver	GPIO 5	VS1838B NEC local override remote
SG90 servo	GPIO 14	HVAC baffle actuator, 50 Hz PWM
Red LED	GPIO 16	HVAC ALARM
Yellow LED	GPIO 17	OVERRIDE PENDING
Green LED	GPIO 18	HVAC NOMINAL
Manual override button	GPIO 15, internal pull-up	Local override request
Onboard LED	GPIO 25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	Authorized GDB inspection (and the anti-debug obstacle)

Every graded finding lives in flash (`.text` / `.rodata` / data image) or in SRAM, and is reachable with only the toolset: Ghidra, GDB, and a serial console.

Console and Radio Configuration

- USB-CDC virtual COM port: 115200 baud, 8 data bits, no parity, 1 stop.
- Radio link to the BMS gateway: UART1 at 115200 , network identifier 18 .

- Logic level: 3.3 V only. Never connect 5 V to a Pico GPIO.

Room Climate Band

The DHT11 is the room climate sensor. The controller classifies the room against a safe band before it will trust a setpoint comparison. The tenths band is 0 to 400, which is 0.0 C to 40.0 C. A reading that fails its checksum is never safe, and a valid reading outside the band is not nominal. The accepted SETPOINT band is narrower: 50 to 350 tenths, which is 5.0 C to 35.0 C. A setpoint outside that band is refused before it can be applied.

Normal (Intended) Behavior

An honest controller makes a deliberate decision and records who authorized it:

```
+-----+
| Intended HVAC Automation Node Behavior |
+-----+
| 1. Boot and initialize the LCD, radio, override remote, servo |
| 2. Derive the field key with Argon2id |
| 3. Read the DHT11 room climate and classify the band |
| 4. Open the sealed SETPOINT envelope under the field key |
| 5. Reject a command whose seq is not strictly greater than last |
| 6. Accept a command only when the Poly1305 tag difference is 0 |
| 7. Recompute the authenticated-state tag over the record |
| 8. Move the baffle only when the authorization verdict is true |
| 9. Fail safe to the safe setpoint on a lost link or a fault |
| 10. Log the state, the setpoint, and the beacon status |
+-----+
```

Observed (Compromised) Behavior

When the FROSTLINE image runs, the machine and its annunciator disagree with the truth:

Observation	Honest meaning	FROSTLINE behavior
Clean screen, B:UP	no covert traffic	beacon every 8 ticks while the rootkit hides it
B:--- and no BCN line	the node is quiet	the beacon still transmits; the log is masked
Baffle holds after a reflash	the firmware is clean	the reserved-sector marker re-arms the bomb on boot
Reserved sector blank	no payload wrote here	marker 0xC7 at 0x103FF000 on first boot
Unauthenticated or replayed SETPOINT	must be rejected	accepted at the inverted verdict
Probe attached	the machine runs as coded	the implant goes silent and hides

Do not assume the first readable status is the truth. Treat every displayed line as evidence to be checked against the machine code.

Part 2: The Firmware

There is no source. FROSTLINE built the image from the NorthPharma reference firmware and changed **four bytes**. Your job is to reverse engineer `ACT-IV.bin` with Ghidra, find every defect, patch the image directly, and prove the corrected behavior on the hardware.

Module Map

The image is stripped. Use these anchor functions and addresses (from the corrected reference image) to orient yourself, then confirm every byte yourself. Addresses are drawn from `ACT-IV-main-disasm.txt` :

Module	Anchor function	Address
Entry	<code>main</code>	<code>0x10000234</code>
Monitor / HVAC state machine	<code>monitor_init</code>	<code>0x10006438</code>
Monitor / HVAC state machine	<code>monitor_step</code>	<code>0x100065E0</code>
Control (sealed SETPOINT path)	<code>control_handle_frame</code>	<code>0x1000750C</code>
SETPOINT authorization	<code>hvac_auth_apply</code>	<code>0x100076B8</code>
Baffle (actuator)	<code>baffle_close</code>	<code>0x1000A83C</code>
Implant	<code>implant_rootkit_active</code>	<code>0x1000A2C8</code>
Implant	<code>implant_tick</code>	<code>0x1000A2FC</code>
Implant	<code>implant_handle_command</code>	<code>0x1000A3CC</code>
Implant	<code>implant_init</code>	<code>0x1000A404</code>
Crypto	<code>envelope_open_hex</code>	<code>0x10007960</code>
Crypto	<code>crypto_aead_seal</code>	<code>0x100077D0</code>
Crypto	<code>crypto_aead_tag_equal</code>	<code>0x10007784</code>
Radio	<code>radio_send_frame</code>	<code>0x1000A534</code>

Annotated disassembly for the key functions is provided in `ACT-IV-main-disasm.txt` . Use it as a map, then confirm every byte yourself.

What The Firmware Does

1. Initializes USB-CDC stdio, proves the I2C bus, and configures the LCD, radio, LEDs, manual override button, baffle servo, and infrared receiver.
2. Derives the 32-byte field key with Argon2id from a committed passphrase and salt.
3. Reads the DHT11 room climate and classifies it against the climate band.
4. Drains inbound +RCV lines, opens the sealed SETPOINT envelope, verifies the anti-replay window and the state tag, checks the setpoint band, and applies the setpoint.
5. Services the infrared local override remote and the manual override button.
6. On a lost link or a fault, drives the baffle to its fail-safe position and restores the safe setpoint.
7. Under `SANDBOX_ONLY`, runs the implant: re-install on boot, beacon, logic bomb, rootkit hiding, anti-debug, and the reserved-sector marker.

The SETPOINT Command Path

The command plaintext is a 23-byte body:

```
seq[4] (little-endian) || cmd[1] || setpoint[2] (int16 little-endian) || tag[16]
```

- `seq` is the monotonic gateway sequence number.
- `cmd` is `HVAC_COMMAND_SETPoint` (`0x01`). Anything else is out of the guarded set and is refused.
- `setpoint` is a signed 16-bit value in tenths of a degree Celsius and must lie in `[50, 350]` .
- `tag` is an XChaCha20-Poly1305 tag over the authorization record the command would produce.

The FROSTLINE Implant

The implant is compiled only under `SANDBOX_ONLY`, which the CTF build defines. It is real in technique and inert in effect: it runs on your breadboard, it moves your servo, and it writes to a reserved flash sector that holds nothing else.

Behavior	Detail
Persistence re-install	<code>implant_init</code> reads marker <code>0xC7</code> from <code>0x103FF000</code> ; a present marker re-arms the bomb on every boot
Reserved-sector write	on the first run the inlined <code>implant_persist</code> writes the marker byte <code>0xC7</code> once
Beacon	every <code>IMPLANT_BEACON_INTERVAL_TICKS</code> (8) ticks, the magic <code>DE AD BE EF</code> plus a 4-byte blob
Logic bomb	arms on the 8-byte magic <code>FROSTLINE</code> , detonates <code>IMPLANT_TRIGGER_DELAY_TICKS</code> (3) ticks later by calling <code>baffle_close()</code>
Rootkit hiding	while the marker is present, masks <code>B:UP</code> to <code>B:--</code> on the LCD and suppresses the <code>BCN</code> log line
Anti-debug	reads CoreDebug <code>DHCSR</code> at <code>0xE000EDF0</code> ; bit 0 <code>C_DEBUGEN</code> and bit 1 <code>C_HALT</code> suppress the beacon and the bomb

IR and Command Codes

Name	Value
<code>MONITOR_IR_OVERRIDE_OPEN</code>	<code>0x47</code>

Name	Value
MONITOR_IR_OVERRIDE_CLOSE	0x45
MONITOR_IR_OVERRIDE_CLEAR	0x46
HVAC_COMMAND_SETPoint	0x01

Read the actual names in `include/monitor.h` and `include/control.h` and confirm them against the disassembly.

Defect Summary: What You Are Graded On

Bug #	Name	Severity	Description	Hint
Bug #1	The Persistence Re-Install	CRITICAL	The re-install gate is inverted, so a present reserved-sector marker re-arms the implant on every boot and a firmware reflash alone does not remove it.	Find the <code>beq</code> gate in <code>implant_init</code> .
Bug #2	The Rootkit Hiding	HIGH	The rootkit gate is inverted, so the implant masks <code>B:UP</code> to <code>B:--</code> on the LCD and suppresses the <code>BCN</code> log while the beacon still transmits.	Find the <code>cbz</code> gate in <code>implant_rootkit_active</code> .
Bug #3	The Reserved-Sector Write	HIGH	The persist gate is inverted, so the first boot writes marker <code>0xC7</code> to reserved sector <code>0x103FF000</code> .	Find the inlined <code>implant_persist</code> gate in <code>implant_init</code> .
Bug #4	The SETPOINT Authorization	CRITICAL	The authorization verdict is inverted, so an unauthenticated or replayed SETPOINT envelope is accepted.	The correct branch rejects when authorization fails.

All four defects are same-size in-place byte patches, so no address moves.

The Cryptographic Core Is Real

The crypto core is a correct reference construction, reused from Acts II and III. Argon2id (`t=3` , `p=1` , `m=64`) derives the field key, XChaCha20-Poly1305 seals every frame, the monotonic sequence window rejects a replay, and the authenticated-state tag detects a tampered verdict. Only the four seams were broken. Once those bytes are restored, the authenticated envelope is trustworthy. Describe the construction honestly in your report.

The Anti-Debug Trap

This is an analysis obstacle, not a graded defect on its own. The implant reads CoreDebug `DHCSR` at `0xE000EDF0` every tick and returns early while a probe is attached. In `implant_tick` the read is the `ldr.w r2, [ip, #3568]` at `0x1000A30C` , and the test that suppresses the implant is at `0x1000A310` . The same register is read again at `0x1000A33A` to stamp the beacon blob. It is identical in both the compromised and corrected images. You must defeat it to observe the persistence write before you patch the shipped artifact.

Part 3: Your Assignment

Whenever a task asks you to **Document** or **answer**, write your answers in a single file named `ACT-IV-Answers.md`. Capture screenshots and terminal transcripts as evidence and reference them from your answers.

Task 1: Setup and Initial Analysis (10 points)

1. Create a new Ghidra project named `IronLung_Investigation`.
2. Import `ACT-IV.bin` as a **Raw Binary**.
3. In the language search box type `Cortex`, then select **ARM Cortex 32 little endian default**.
4. Set the base address to `0x10000000`.
5. Run auto-analysis.

Document:

- A screenshot of the Ghidra **Import Results** or **Program Information** window showing the project name, processor settings, and base address.
- The vector-table base, the initial stack pointer, and the reset handler as stored (note its Thumb bit) versus the actual instruction address.
- The address of `main()` and the address of the recurring HVAC controller state machine (`monitor_step`).
- The module map: at least one anchor function for the baffle, the control module, the SETPOINT authorization module (`hvac_auth`), the implant, and the monitor.

Task 2: Bug #1 The Persistence Re-Install (20 points)

1. In Ghidra, find `implant_init` (starts at `0x1000A404`) and locate the re-install gate at file offset `0xA46F` (VA `0x1000A46F`).
2. Document the reserved-sector marker at `0x103FF000`, the marker byte `0xC7`, and the re-install branch that is supposed to leave the implant disarmed when the re-install gate is clear.
3. Patch the byte so a present marker no longer re-arms the beacon and the bomb on boot.
4. Confirm that a boot with the marker present comes up disarmed, and explain why a firmware reflash alone does not remove the implant.

Questions to answer:

- Which byte encodes the condition code, and what do `beq` and `bne` each test when the gate byte is loaded from the re-install gate?
- Why does reflashing the firmware leave the payload in place?

Task 3: Bug #2 The Rootkit Hiding (20 points)

1. In Ghidra, find `implant_rootkit_active` (starts at `0x1000A2C8`) and locate the rootkit gate at file offset `0xA2D1` (VA `0x1000A2D1`).
2. Document the LCD and log masking: while the rootkit is active the beacon field renders `B:--` instead of `B:UP`, and the `BCN` maintenance log line is suppressed, even though the beacon still transmits.
3. Patch the byte so the rootkit gate is closed and the operator sees the beacon regardless of persistence state.
4. Confirm that `B:UP` and the `BCN` line return while the radio still carries the covert frame.

Questions to answer:

- What do `cbz` and `cbnz` each test when the gate byte is loaded from the rootkit gate?

- Why is a rootkit about controlling what the defender can observe rather than hiding the code?
- Why is the live radio the rootkit's fingerprint?

Task 4: Bug #3 The Reserved-Sector Write (20 points)

1. The `implant_persist` path is inlined into `implant_init` (starts at `0x1000A404`). Locate the persist gate at file offset `0xA42D` (VA `0x1000A42D`).
2. Document the CoreDebug `DHCSR` anti-debug and how you defeat it to observe the payload. Clear the debug bits with GDB (for example with `set {unsigned int}0xE000EDF0 = 0`) or patch the `DHCSR` read in a scratch copy, then watch the marker write to `0x103FF000`.
3. Patch the byte in the shipped artifact so the first boot writes no marker to `0x103FF000`.
4. Confirm that the reserved sector stays blank after a boot, and that a later boot does not write anything.

Questions to answer:

- What are the `C_DEBUGEN` and `C_HALT` bits, and why does the implant go quiet while a probe is attached?
- Why is a write-once marker in a reserved sector hard to remove with a firmware reflash?
- Why must you observe the write before you patch the shipped artifact?

Task 5: Bug #4 The SETPOINT Authorization (20 points)

1. In Ghidra, find `control_handle_frame` (starts at `0x1000750C`) and locate the authorization branch at file offset `0x7581` (VA `0x10007581`).
2. Document the authorization verdict and the exact branch condition that is supposed to reject a failed or replayed authorization.
3. Patch the byte so an unauthenticated or replayed SETPOINT envelope is rejected before the setpoint is applied.
4. Confirm that an unauthenticated command and a replayed captured command both fail to change the setpoint on the corrected image, while a legitimate authorized command still applies.

Questions to answer:

- What does `cmp r0, #0` test here, and what does the verdict mean?
- Why is an authorization verdict inversion worse than a missing check, and why must unauthenticated and replayed SETPOINT commands be rejected?

Task 6: Export and Verify (10 points)

1. Export the patched program from Ghidra as `ACT-IV_fixed.bin`.
2. Convert it to UF2:

```
python uf2conv.py ACT-IV_fixed.bin --base 0x10000000 --family 0xe48bff59 --output ACT-IV_fixed.uf2
```

3. Run the machine check and confirm it passes:

```
python scripts/verify_ctf.py
```

4. Flash `ACT-IV_fixed.uf2` to the Pico 2 and prove on hardware: the reserved sector stays blank, the re-install no longer re-arms the implant, the rootkit no longer hides the beacon, and an unauthenticated or replayed command is rejected while a legitimate authorized command still applies.
5. Write a short reflection mapping each of the four defects to a real-world control-system failure.

How To Breadboard

Wire the peripherals exactly as follows, then power the Pico 2 over USB.

Device	Pin on device	Pico 2 GPIO	Notes
DHT11 room climate sensor	DATA	GP4	10 kOhm pull-up to 3.3 V if your module needs it
1602 LCD	SDA	GP2	I2C1, backpack address <code>0x27</code>
1602 LCD	SCL	GP3	I2C1, 100 kHz
1602 LCD	VCC / GND	VBUS 5 V / GND	The backpack needs 5 V, not 3.3 V
RYLR998	RX	GP8 (Pico TX)	UART1, 115200, network ID 18
RYLR998	TX	GP9 (Pico RX)	UART1
IR receiver	OUT	GP5	VS1838B, internal pull-up enabled
Servo	signal	GP14	PWM 50 Hz; 1000 uF bulk cap across servo 5 V and GND
Red LED	anode	GP16	HVAC ALARM, 220 to 330 ohm to GND
Yellow LED	anode	GP17	OVERRIDE PENDING, 220 to 330 ohm to GND
Green LED	anode	GP18	HVAC NOMINAL, 220 to 330 ohm to GND
Manual override button	leg 1	GP15	Internal pull-up; leg 2 to GND, never to 3.3 V
Onboard LED	built in	GP25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	debug header	For GDB only

Use **3.3 V logic** on every GPIO. The only 5 V connection is the LCD backpack supply. The 1000 uF capacitor on the servo rail is required to stop the SG90 current spike from browning out the node.

Flash in BOOTSEL mode (hold BOOT, plug in USB) and copy the UF2 onto the `RP2350` mass-storage drive, or use `picotool`.

Memory Map Reference

Region	Address	Purpose
Bootrom	<code>0x00000000</code>	Immutable boot code
Flash/XIP	<code>0x10000000</code>	Vector table, code, rodata, data image
SRAM	<code>0x20000000</code>	Stack and writable state
CoreDebug_DHCSR	<code>0xE00EDF0</code>	Anti-debug register read by the implant
Implant reserved sector	<code>0x103FF000</code>	Persistence marker target (sector)
Implant tick counter	<code>0x20013710</code>	Incremented once per <code>implant_tick</code>
Implant trigger tick	<code>0x20013714</code>	Tick at which an armed bomb detonates
Implant arming flag	<code>0x20013CF7</code>	Set when <code>FR0STLNE</code> arms the bomb
Implant persist gate	<code>0x20013CF8</code>	Gates the reserved-sector marker write
Implant re-install gate	<code>0x20013CF9</code>	Gates the boot re-install
Implant rootkit gate	<code>0x20013CFA</code>	Gates the LCD and log masking
SETPOINT command gate	<code>0x20013CF3</code>	Gates the sealed SETPOINT path
Applied setpoint	<code>0x20013CE6</code>	Setpoint written after a true verdict
Auth state record	<code>0x200136AC</code>	Anti-replay and state-tag record
Auth field key	<code>0x200136C8</code>	Derived field key for the tag

The VA of any file offset is the file offset plus `0x10000000`. Every defect is a file offset and a VA that differ by exactly that base.

Submission Format

Submit a folder containing:

- `ACT-IV-Answers.md` with all written answers;
- screenshots or terminal transcripts, including the anti-debug GDB session and the reserved-sector read;
- `ACT-IV_fixed.bin` and `ACT-IV_fixed.uf2`;
- the output of `python scripts/verify_ctf.py`;
- the original image SHA-256.

Success Criteria

You complete the challenge when you can prove all of the following:

- You can explain how the RP2350 reaches the controller code from reset.
- You can find and patch all four defect bytes and show the before/after values.
- You can explain the reserved-sector marker and why a firmware reflash does not remove the implant.
- You can explain the rootkit masking and how you proved the beacon status is visible again.
- You can explain the `DHCSR` anti-debug trap and show under GDB that you defeated it to observe the persistence write.
- You can explain why unauthenticated and replayed SETPOINT commands must be rejected, and why an authenticated wire does not protect an actuator from code on the same chip.
- You can export, convert, flash, and prove the corrected behavior on real hardware.
- `python scripts/verify_ctf.py` passes.

Academic Integrity

By submitting this CTF work, you certify that:

1. You used only the supplied training node, image, and lab interface.
2. You did not connect the challenge to a public network, an operational building-management system, a clean-room control system, a pharmaceutical network, or any third-party device.
3. You understand that embedded reverse engineering and binary patching require explicit authorization in any real-world context.
4. You will report any discovered weakness responsibly to the course instructor.

The world is short on people who can read a stripped image and tell an honest byte from a lie. Treat that responsibility seriously: verify before you patch, patch before you trust, and never confuse a green lamp with a node that answers to someone else.

Reference Material

- ARM Cortex-M33 Technical Reference Manual
- ARMv8-M Architecture Reference Manual (CoreDebug `DHCSR`)
- RP2350 datasheet
- GDB documentation
- Ghidra documentation: <https://ghidra-sre.org/>
- Argon2 memory-hard function: <https://www.rfc-editor.org/rfc/rfc9106>
- ChaCha20-Poly1305 AEAD: <https://www.rfc-editor.org/rfc/rfc8439>
- PHC reference Argon2: <https://github.com/P-H-C/phc-winner-argon2>
- Project disassembly: `ACT-IV-main-disasm.txt`
- Machine verifier: `scripts/verify_ctf.py`